UNIVERSIDAD DEL MAGDALENA— FACULTAD DE INGENIERÍA

Estructuras de Datos II

Complejidad Computacional, Técnicas Algorítmicas y Diseño de Algoritmos en Java

Paper Técnico – Workshop Syste+

2026 - JAIRO BLANCO

Área: Algoritmos y Estructuras de Datos
Nivel: Pregrado – Ingeniería de Sistemas

Resumen

Este documento presenta un análisis técnico y didáctico de los fundamentos de la complejidad computacional y las principales técnicas de diseño algorítmico, implementadas en Java. Se abordan la medición experimental de tiempo y memoria, la recursividad, los paradigmas *Divide y Vencerás*, *Backtracking*, *Ramificación y Poda*, algoritmos voraces (*Greedy*), heurísticas y programación dinámica. Para cada técnica se estudia su estructura lógica, sus complejidades teóricas asociadas y se presentan plantillas de abstracción que permiten instanciar la lógica subyacente ante distintos problemas. Adicionalmente, se introduce el concepto de tabla hash y funciones de hashing como estructura auxiliar fundamental. El trabajo busca consolidar la transición conceptual desde Estructuras de Datos I hacia el análisis y diseño eficiente de algoritmos.

Índice

1. Complejidad Computacional: Medición Práctica **3**
1.1. Contexto y Motivación . . . 3
1.2. Medición de Tiempo con `System.nanoTime()` . . . 3
1.3. Medición de Memoria . . . 4

2. Recursividad **4**
2.1. Definición y Estructura . . . 4
2.2. Profundidad de Pila y Stack Overflow . . . 5
2.3. Recursividad de Cola (Tail Recursion) . . . 5

3. Técnicas de Diseño Algorítmico **6**
3.1. Divide y Vencerás . . . 6
3.1.1. Principio . . . 6
3.1.2. Complejidad: Teorema Maestro . . . 6
3.2. Backtracking y Árbol de Decisión . . . 7
3.2.1. Principio . . . 7
3.2.2. Árbol de Decisión . . . 7
3.3. Ramificación y Poda (Branch & Bound) . . . 8
3.3.1. Principio . . . 8
3.4. Algoritmos Voraces (Greedy) . . . 9
3.4.1. Principio . . . 9
3.5. Heurísticas . . . 10
3.5.1. Principio . . . 10
3.6. Programación Dinámica . . . 11
3.6.1. Principio . . . 11

4. Plantillas de Abstracción Algorítmica **12**
4.1. Plantilla: Divide y Vencerás . . . 12
4.2. Plantilla: Backtracking . . . 13
4.3. Plantilla: Greedy . . . 13
4.4. Plantilla: Programación Dinámica . . . 14

5. Tabla Hash **14**
5.1. Función de Hashing 14
5.1.1. Ejemplo: Hash por División y Multiplicación 15
5.1.2. Colisiones y Resolución 15

6. Conclusiones **15**

1 Complejidad Computacional: Medición Práctica

1.1 Contexto y Motivación

La **complejidad computacional** estudia los recursos que un algoritmo consume en función del tamaño de su entrada. Aunque la notación asintótica ($\mathcal{O}$, Ω, Θ) nos provee una caracterización teórica, la medición experimental permite validar hipótesis, detectar constantes ocultas y entender comportamientos en contextos reales con JVM, cache y otros factores.

Notación Asintótica

Sea $f(n)$ el tiempo de ejecución de un algoritmo. Decimos que $f(n) \in \mathcal{O}(g(n))$ si existen constantes $c > 0$ y $n_0 \geq 0$ tales que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$. La notación Ω expresa cota inferior y Θ acotamiento ajustado por ambos lados.

1.2 Medición de Tiempo con `System.nanoTime()`

En Java, `System.nanoTime()` retorna el tiempo en nanosegundos del reloj de alta resolución del sistema. A diferencia de `System.currentTimeMillis()`, no está sujeto a cambios del reloj del sistema y tiene mayor precisión para medir intervalos cortos.

```java
public class MedicionTiempo {
    public static void main(String[] args) {
        int[] datos = generarArreglo(100_000);

        // Calentar la JVM (warm-up)
        for (int i = 0; i < 3; i++) bubbleSort(datos.clone());

        long inicio = System.nanoTime();
        bubbleSort(datos.clone());
        long fin = System.nanoTime();

        double ms = (fin - inicio) / 1_000_000.0;
        System.out.printf("Tiempo: %.3f ms%n", ms);
    }

    static void bubbleSort(int[] a) {
        for (int i = 0; i < a.length - 1; i++)
            for (int j = 0; j < a.length - i - 1; j++)
                if (a[j] > a[j+1]) { int t = a[j]; a[j] = a[j+1]; a
                    [j+1] = t; }
    }
}
```

Listing 1: Medición de tiempo de ejecución en Java

Sesgos a Considerar

La JVM realiza compilación JIT (*Just-In-Time*) que optimiza el código en ejecución. El primer run suele ser más lento. Siempre se recomienda hacer un *warm-up* de al menos 3–5 iteraciones antes de tomar mediciones y promediar

varios ensayos.

1.3 Medición de Memoria

El consumo de memoria puede estimarse usando la API de `Runtime`:

```java
public class MedicionMemoria {
    static long memoriaUsada() {
        Runtime rt = Runtime.getRuntime();
        rt.gc(); // solicitar GC
        return rt.totalMemory() - rt.freeMemory();
    }

    public static void main(String[] args) {
        long antes = memoriaUsada();
        int[] arreglo = new int[1_000_000];
        long despues = memoriaUsada();
        System.out.printf("Memoria usada: %.2f MB%n",
            (despues - antes) / 1_048_576.0);
    }
}
```

Listing 2: Estimación de memoria usada

Complejidades de referencia

Clase	Ejemplo	$n = 10^6$, aprox.
$\mathcal{O}(1)$	Acceso a arreglo	Constante
$\mathcal{O}(\log n)$	Búsqueda binaria	≈ 20 ops
$\mathcal{O}(n)$	Recorrido lineal	10^6 ops
$\mathcal{O}(n \log n)$	MergeSort	$\approx 2 \times 10^7$
$\mathcal{O}(n^2)$	BubbleSort	10^{12} ops
$\mathcal{O}(2^n)$	Subconjuntos	inviable

2 Recursividad

2.1 Definición y Estructura

Un algoritmo es **recursivo** cuando su definición incluye una o más llamadas a sí mismo con un subproblema estrictamente más pequeño. Todo algoritmo recursivo debe cumplir:

1. **Caso base**: condición de parada que no invoca recursión.
2. **Caso recursivo**: reducción hacia el caso base.
3. **Progreso**: garantía de que la entrada se acerca al caso base.

2.2 Profundidad de Pila y Stack Overflow

Cada llamada recursiva apila un *frame* en la pila de llamadas (*call stack*). Si la profundidad es excesiva (tipicamente > 500–10,000 dependiendo de la JVM), se lanza una `StackOverflowError`. La JVM de Java asigna un tamaño fijo a la pila de cada hilo (por defecto ≈ 512 KB a 1 MB).

```
// Recursivo: O(n) tiempo, O(n) espacio en pila
long factorialRec(int n) {
    if (n <= 1) return 1;           // Caso base
    return n * factorialRec(n - 1); // Caso recursivo
}

// Iterativo: O(n) tiempo, O(1) espacio
long factorialIter(int n) {
    long resultado = 1;
    for (int i = 2; i <= n; i++) resultado *= i;
    return resultado;
}
```

Listing 3: Factorial: recursivo vs iterativo

2.3 Recursividad de Cola (Tail Recursion)

En la **recursividad de cola**, la llamada recursiva es la *última* operación del método. Algunos compiladores/runtimes (no la JVM por defecto) pueden optimizarla y reutilizar el mismo frame, reduciendo el uso de pila a $\mathcal{O}(1)$.

```
// Fibonacci ingenuo: O(2^n) tiempo, O(n) pila
int fibRec(int n) {
    if (n <= 1) return n;
    return fibRec(n-1) + fibRec(n-2); // NO tail: dos llamadas
}

// Fibonacci con acumulador (tail-style): O(n) tiempo
int fibTail(int n, int a, int b) {
    if (n == 0) return a;
    return fibTail(n-1, b, a+b); // tail
}
// Llamada: fibTail(n, 0, 1)
```

Listing 4: Fibonacci: múltiple vs tail

Costos Computacionales de la Recursividad

- **Tiempo**: Igual al número de llamadas recursivas generadas. El árbol de recursión tiene $\mathcal{O}(2^n)$ nodos para Fibonacci ingenuo.
- **Espacio**: $\mathcal{O}(d)$ donde d es la profundidad máxima del árbol de recursión.
- **Overhead**: Cada frame incluye parámetros, variables locales y dirección de retorno. En Java esto es significativo comparado con la iteración.

3 Técnicas de Diseño Algorítmico

3.1 Divide y Vencerás

3.1.1 Principio

El paradigma **Divide y Vencerás** (*Divide and Conquer*) estructura la solución en tres etapas:

1. **Dividir**: Partir el problema de tamaño n en k subproblemas de tamaño n/b.
2. **Conquistar**: Resolver cada subproblema recursivamente.
3. **Combinar**: Fusionar las soluciones parciales en la solución global.

3.1.2 Complejidad: Teorema Maestro

Si la recurrencia tiene la forma $T(n) = a \cdot T(n/b) + f(n)$ con $a \geq 1$, $b > 1$:

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{si } f(n) = \mathcal{O}(n^{\log_b a - \varepsilon}) \\ \Theta(n^{\log_b a} \log n) & \text{si } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{si } f(n) = \Omega(n^{\log_b a + \varepsilon}) \end{cases}$$

MergeSort: $T(n) = 2T(n/2) + \mathcal{O}(n) \Rightarrow \mathcal{O}(n \log n)$.

```java
void mergeSort(int[] arr, int l, int r) {
    if (l >= r) return;
    int mid = (l + r) / 2;
    mergeSort(arr, l, mid);       // Dividir izquierda
    mergeSort(arr, mid+1, r);     // Dividir derecha
    merge(arr, l, mid, r);        // Combinar
}

void merge(int[] arr, int l, int mid, int r) {
    int n1 = mid - l + 1, n2 = r - mid;
    int[] L = Arrays.copyOfRange(arr, l, mid+1);
    int[] R = Arrays.copyOfRange(arr, mid+1, r+1);
    int i=0, j=0, k=l;
    while (i<n1 && j<n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i<n1) arr[k++] = L[i++];
    while (j<n2) arr[k++] = R[j++];
}
```

Listing 5: MergeSort en Java

Divide y Vencerás

- **MergeSort**: $\mathcal{O}(n \log n)$ tiempo, $\mathcal{O}(n)$ espacio auxiliar.
- **QuickSort**: $\mathcal{O}(n \log n)$ esperado, $\mathcal{O}(n^2)$ peor caso. $\mathcal{O}(\log n)$ espacio en pila (promedio).
- **Búsqueda Binaria**: $\mathcal{O}(\log n)$ tiempo, $\mathcal{O}(1)$ espacio.

3.2 Backtracking y Árbol de Decisión

3.2.1 Principio

El **backtracking** es una estrategia de búsqueda exhaustiva con retroceso. Se exploran decisiones incrementales; si una decisión lleva a un estado inválido o no prometedor, se *deshace* (*retrocede*) y se prueba la siguiente alternativa.

3.2.2 Árbol de Decisión

El **árbol de decisión** (*decision tree*) es la estructura auxiliar conceptual que modela el espacio de búsqueda del backtracking:

- **Nodo raíz**: Estado inicial, sin ninguna decisión tomada.
- **Nodos internos**: Estado parcial después de k decisiones.
- **Aristas**: Cada arista representa una elección concreta (p.ej. colocar la reina en la columna j).
- **Hojas**: Solución completa (éxito) o estado muerto (fallo por restricción).
- **Poda**: Una rama se corta cuando se detecta anticipadamente que no puede llevar a una solución válida, evitando explorar subárboles enteros.

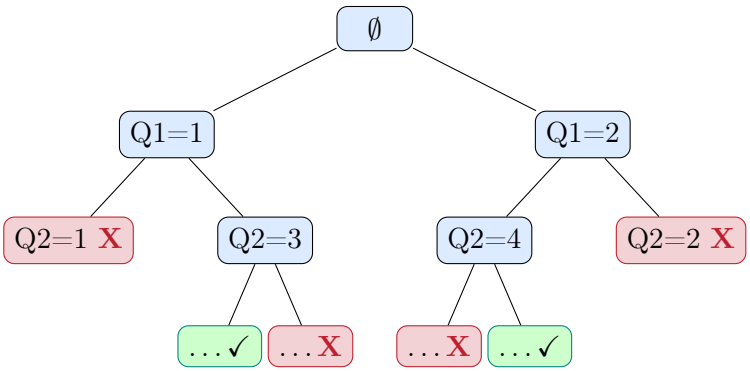

Figura 1: Árbol de decisión para el problema de las N-Reinas. Los nodos rojos son podados por conflicto; los verdes son soluciones válidas.

```java
public class NReinas {
    int N;
    int[] col; // col[i] = columna donde esta la reina en fila i

    void resolver(int fila) {
        if (fila == N) { imprimirSolucion(); return; }
        for (int c = 0; c < N; c++) {
            if (esValido(fila, c)) {
                col[fila] = c;          // Decidir
                resolver(fila + 1);     // Explorar
                // Retroceso implicito: col[fila] se sobreescribe
            }
        }
    }
```

```java

    boolean esValido(int fila, int c) {
        for (int i = 0; i < fila; i++) {
            if (col[i] == c) return false;
            if (Math.abs(col[i] - c) == Math.abs(i - fila)) return
                false;
        }
        return true;
    }
}
```

Listing 6: N-Reinas con Backtracking

Backtracking

- **Peor caso sin poda**: $\mathcal{O}(k^n)$ donde k es el número de alternativas por nivel y n la profundidad.
- **N-Reinas**: $\mathcal{O}(n!)$ sin poda, considerablemente mejor con poda.
- **Espacio**: $\mathcal{O}(n)$ en pila (profundidad del árbol de decisión).
- **Efectividad**: Dependiente de la calidad de la función de poda. Una buena función de poda puede reducir el árbol de $\mathcal{O}(k^n)$ a polinomial en práctica.

3.3 Ramificación y Poda (Branch & Bound)

3.3.1 Principio

La **ramificación y poda** extiende el backtracking con una función de **cota** (*bound*): en cada nodo se calcula una cota optimista de la mejor solución alcanzable desde ese subárbol. Si la cota no puede mejorar la mejor solución encontrada hasta el momento, el subárbol se poda.

Diferencia con Backtracking

Mientras el backtracking poda sólo por **factibilidad** (violación de restricciones), Branch & Bound poda por **optimalidad**: se descartan ramas que, aunque produzcan soluciones válidas, no pueden mejorar el óptimo conocido.

```java
class Nodo {
    int nivel, beneficio, peso;
    double cotaSuperior; // bound optimista
}

double cotaSuperior(Nodo u, int n, int W, Item[] items) {
    if (u.peso >= W) return 0;
    double bound = u.beneficio;
    int pesoTot = u.peso;
    int j = u.nivel + 1;
    while (j < n && pesoTot + items[j].peso <= W) {
        pesoTot += items[j].peso;
        bound += items[j].valor;
```

```
        j++;
    }
    if (j < n) bound += (W - pesoTot) * (double)items[j].valor /
        items[j].peso;
    return bound;
}
```

Listing 7: Mochila 0/1 con Branch and Bound (esquema)

Branch & Bound

- **Peor caso**: $\mathcal{O}(2^n)$ (sin poda efectiva, degrada a backtracking).
- **Caso promedio**: Muy dependiente de la calidad de la función de cota.
- **Mochila 0/1**: Pseudo-polinomial $\mathcal{O}(nW)$ con DP; B&B es competitivo en práctica.

3.4 Algoritmos Voraces (Greedy)

3.4.1 Principio

Un algoritmo **voraz** construye la solución incrementalmente, eligiendo en cada paso la opción *localmente óptima* con la esperanza de alcanzar la *solución global óptima*. No retrocede: cada elección es definitiva.

Condiciones para Corrección

Un algoritmo greedy produce la solución óptima si y solo si el problema exhibe:

1. **Propiedad greedy**: La elección localmente óptima es parte de la solución globalmente óptima.
2. **Subestructura óptima**: La solución óptima contiene soluciones óptimas de sus subproblemas.

```
int[] dijkstra(int[][] grafo, int src) {
    int n = grafo.length;
    int[] dist = new int[n];
    boolean[] visitado = new boolean[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[1]-b
        [1]);
    pq.offer(new int[]{src, 0});

    while (!pq.isEmpty()) {
        int[] u = pq.poll();
        int nodo = u[0];
        if (visitado[nodo]) continue;
        visitado[nodo] = true;

        for (int v = 0; v < n; v++) {
            if (grafo[nodo][v] > 0 && !visitado[v]) {
```

```
                    int nuevaDist = dist[nodo] + grafo[nodo][v];
                    if (nuevaDist < dist[v]) {
                        dist[v] = nuevaDist;
                        pq.offer(new int[]{v, dist[v]});
                    }
                }
            }
        }
    return dist;
}
```

Listing 8: Algoritmo de Dijkstra (Greedy)

Algoritmos Greedy comunes

Algoritmo	Tiempo	Espacio
Dijkstra (heap)	$\mathcal{O}((V+E)\log V)$	$\mathcal{O}(V)$
Kruskal (MST)	$\mathcal{O}(E \log E)$	$\mathcal{O}(V)$
Prim (MST)	$\mathcal{O}((V+E)\log V)$	$\mathcal{O}(V)$
Selección de actividades	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Huffman coding	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$

3.5 Heurísticas

3.5.1 Principio

Una **heurística** es una estrategia práctica que sacrifica optimalidad o completitud garantizada para obtener soluciones *suficientemente buenas* en tiempos razonables. Son esenciales cuando el espacio de búsqueda es intratable ($\mathcal{NP}$-difícil).

Heurística vs Algoritmo Exacto

Un algoritmo exacto garantiza la solución óptima pero puede ser exponencial. Una heurística ofrece soluciones aproximadas en tiempo polinomial, sin garantía de optimalidad.

Entre las heurísticas más estudiadas se encuentran: **Hill Climbing**, **Simulated Annealing**, **Algoritmos Genéticos** y **Búsqueda Tabú**. Para el **Problema del Viajante (TSP)**, la heurística del vecino más cercano produce rutas de calidad razonable en $\mathcal{O}(n^2)$.

```
int[] tspVecinoMasCercano(double[][] dist, int inicio) {
    int n = dist.length;
    boolean[] visitado = new boolean[n];
    int[] ruta = new int[n+1];
    ruta[0] = inicio;
    visitado[inicio] = true;

    for (int paso = 1; paso < n; paso++) {
        int actual = ruta[paso-1];
        int mejorVecino = -1;
        double mejorDist = Double.MAX_VALUE;
```

```
        for (int j = 0; j < n; j++) {
            if (!visitado[j] && dist[actual][j] < mejorDist) {
                mejorDist = dist[actual][j];
                mejorVecino = j;
            }
        }
        ruta[paso] = mejorVecino;
        visitado[mejorVecino] = true;
    }
    ruta[n] = inicio; // regresar al origen
    return ruta;
}
```

Listing 9: Vecino más cercano para TSP (heurística)

Heurísticas

- **Vecino más cercano (TSP)**: $\mathcal{O}(n^2)$ tiempo, $\mathcal{O}(n)$ espacio. Calidad: $\approx$ 20–25 % sobre el óptimo en promedio.
- **Simulated Annealing**: $\mathcal{O}(k \cdot n)$ donde k es el número de iteraciones de enfriamiento. Sin garantía formal de optimalidad.

3.6 Programación Dinámica

3.6.1 Principio

La **programación dinámica** (DP) resuelve problemas mediante la descomposición en subproblemas solapados, almacenando sus resultados para evitar recomputaciones. Requiere:

1. **Subestructura óptima**: La solución óptima se construye desde soluciones óptimas de subproblemas.
2. **Subproblemas solapados**: Los mismos subproblemas se resuelven múltiples veces sin DP.

Existen dos enfoques: **top-down** (memoización) y **bottom-up** (tabulación).

```
int mochila(int W, int[] pesos, int[] valores, int n) {
    // dp[i][w] = maximo valor con i items y capacidad w
    int[][] dp = new int[n+1][W+1];
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i-1][w]; // no tomar item i
            if (pesos[i-1] <= w)
                dp[i][w] = Math.max(dp[i][w],
                    dp[i-1][w-pesos[i-1]] + valores[i-1]);
        }
    }
    return dp[n][W];
}
```

Listing 10: Problema de la Mochila 0/1: DP Bottom-Up

Programación Dinámica

Problema	Tiempo DP	Espacio
Mochila 0/1	$\mathcal{O}(nW)$	$\mathcal{O}(nW)$
LCS	$\mathcal{O}(mn)$	$\mathcal{O}(mn)$
Fibonacci (memo)	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Floyd-Warshall	$\mathcal{O}(V^3)$	$\mathcal{O}(V^2)$
Edit Distance	$\mathcal{O}(mn)$	$\mathcal{O}(mn)$

DP vs Divide y Vencerás

Ambos descomponen el problema en subproblemas. La diferencia: Divide y Vencerás resuelve subproblemas *independientes*; DP resuelve subproblemas *solapados* y guarda los resultados. Aplicar DP a subproblemas independientes introduce overhead inútil.

4 Plantillas de Abstracción Algorítmica

Un **patrón de diseño algorítmico** puede representarse como una **plantilla de abstracción**: una estructura lógica genérica con *huecos* que deben instanciarse concretamente según el problema. Esta perspectiva separa la *mecánica invariante* de la técnica (la lógica que siempre se mantiene) de los *parámetros variables* que dependen del dominio del problema.

La clave pedagógica es que al enfrentar un nuevo problema, el diseñador debe reconocer la técnica aplicable, identificar la plantilla y luego **abstraer** los elementos propios del problema para rellenar cada hueco.

4.1 Plantilla: Divide y Vencerás

```
// === PLANTILLA ABSTRACTA ===
// INVARIANTE: dividir -> resolver -> combinar
T divideVenceras(Problema p) {
    if (esCasoBase(p))          // <-- INSTANCIAR: criterio de trivialidad
        return resolverDirecto(p); // <-- INSTANCIAR: solucion trivial

    List<Problema> subproblemas = dividir(p); // <-- INSTANCIAR: estrategia de division

    List<T> soluciones = new ArrayList<>();
    for (Problema sp : subproblemas)
        soluciones.add(divideVenceras(sp)); // RECURSION (INVARIANTE)

    return combinar(soluciones, p);  // <-- INSTANCIAR: estrategia de fusion
}

// Para MergeSort:
```

```
//    esCasoBase(p)   ->  arr.length <= 1
//    dividir(p)      ->  particionar en mitades
//    combinar(s, p)  ->  merge ordenado
```

Listing 11: Plantilla abstracta: Divide y Vencerás

4.2 Plantilla: Backtracking

```
// === PLANTILLA ABSTRACTA ===
// INVARIANTE: explorar -> verificar -> retroceder
void backtrack(Estado e, int nivel) {
    if (esSolucionCompleta(e)) {  // <-- INSTANCIAR
        registrarSolucion(e);
        return;
    }

    for (Decision d : decisionesDisponibles(e)) { // <-- INSTANCIAR
        if (esValida(e, d)) {       // <-- INSTANCIAR: restricciones
            aplicar(e, d);          // avanzar
            backtrack(e, nivel+1); // RECURSION (INVARIANTE)
            deshacer(e, d);         // RETROCESO (INVARIANTE)
        }
    }
}

// Para N-Reinas:
//    esSolucionCompleta ->  nivel == N
//    decisionesDisponibles -> columnas 0..N-1
//    esValida -> no hay conflictos entre reinas
//    aplicar/deshacer -> colocar/quitar reina en col[nivel]
```

Listing 12: Plantilla abstracta: Backtracking

4.3 Plantilla: Greedy

```
// === PLANTILLA ABSTRACTA ===
// INVARIANTE: seleccionar localmente optimo hasta agotar
    candidatos
Solucion greedy(Set<Candidato> candidatos) {
    Solucion s = new Solucion();
    while (!candidatos.isEmpty()) {
        Candidato c = seleccionarMejor(candidatos); // <--
            INSTANCIAR: criterio greedy
        candidatos.remove(c);
        if (esFactible(s, c))     // <-- INSTANCIAR: restriccion
            s.agregar(c);
    }
    return s;
}

// Para seleccion de actividades:
//    seleccionarMejor -> actividad con menor tiempo de finalizacion
//    esFactible -> inicio >= ultimo fin registrado
```

Listing 13: Plantilla abstracta: Algoritmo Voraz

4.4 Plantilla: Programación Dinámica

```
// === PLANTILLA ABSTRACTA ===
// INVARIANTE: rellenar tabla desde casos base hacia el objetivo
T programacionDinamica(Entrada entrada) {
    T[] dp = inicializar(tamano(entrada)); // <-- INSTANCIAR: dimension y tipo
    establecerCasosBase(dp);               // <-- INSTANCIAR: valores iniciales

    for (int i : ordenDeComputo(entrada)) { // <-- INSTANCIAR: orden de llenado
        dp[i] = transicion(dp, i, entrada); // <-- INSTANCIAR: ecuacion de recurrencia
    }

    return dp[objetivo()]; // <-- INSTANCIAR: posicion de la respuesta
}

// Para Fibonacci:
//   dp[i] = dp[i-1] + dp[i-2]; casos base: dp[0]=0, dp[1]=1
// Para Mochila:
//   dp[i][w] = max(dp[i-1][w], dp[i-1][w-pi] + vi)
```

Listing 14: Plantilla abstracta: DP Bottom-Up

La ventaja de esta perspectiva es que el diseñador **razona en dos capas**: primero reconoce la técnica (instanciando la plantilla) y luego concreta cada parámetro según el dominio. Esto reduce los errores conceptuales y facilita la transferencia de conocimiento entre problemas distintos.

5 Tabla Hash

Tabla Hash

Una **tabla hash** (o tabla de dispersión) es una estructura de datos que implementa un **mapa asociativo**: dado una clave k, permite almacenar y recuperar el valor asociado v en tiempo promedio $\mathcal{O}(1)$. Internamente, mantiene un arreglo de *buckets* y utiliza una **función de hashing** para mapear claves a índices del arreglo.

5.1 Función de Hashing

Una **función hash** $h : U \to \{0, 1, \ldots, m-1\}$ transforma una clave del universo U en un índice dentro del arreglo de tamaño m. Una buena función hash debe ser:

- **Determinista**: la misma clave siempre produce el mismo índice.
- **Uniforme**: distribuye las claves homogéneamente para minimizar colisiones.
- **Eficiente**: $\mathcal{O}(1)$ de cómputo.

5.1.1 Ejemplo: Hash por División y Multiplicación

$$h(k) = k \bmod m \quad \text{(división, simple)}$$
$$h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor, \quad A \approx 0{,}6180 \quad \text{(multiplicación, Knuth)}$$

```java
// Hash nativo en Java
Map<String, Integer> mapa = new HashMap<>();
mapa.put("clave", 42);
int valor = mapa.get("clave"); // O(1) promedio

// Funcion hash custom para strings
int hashString(String s, int m) {
    int hash = 0;
    for (char c : s.toCharArray())
        hash = (hash * 31 + c) % m;
    return hash;
}
```

Listing 15: Hash en Java: uso de HashMap y función custom

5.1.2 Colisiones y Resolución

Cuando $h(k_1) = h(k_2)$ con $k_1 \neq k_2$, ocurre una **colisión**. Las estrategias más comunes son:

- **Encadenamiento (chaining)**: Cada bucket contiene una lista enlazada.
- **Sondeo lineal (open addressing)**: Si el bucket está ocupado, se prueba $h(k) + 1, h(k) + 2, \ldots$

Tabla Hash

Operación	Promedio	Peor caso
Inserción	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Búsqueda	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Eliminación	$\mathcal{O}(1)$	$\mathcal{O}(n)$

El peor caso ocurre cuando todas las claves colisionan en el mismo bucket (degrada a lista enlazada). Con un buen factor de carga $\alpha = n/m < 0{,}75$ y una función hash uniforme, las colisiones son mínimas.

6 Conclusiones

El análisis de las técnicas algorítmicas presentadas revela una jerarquía conceptual clara. La **recursividad** es el mecanismo subyacente de la mayoría de las técnicas avanzadas. **Divide y Vencerás** la estructura de forma sistemática para problemas independientes; el **backtracking** la convierte en exploración exhaustiva con retroceso controlado por el árbol de decisión; **Branch & Bound** la refina con

criterios de optimalidad; la **programación dinámica** la combina con memorización para eliminar la recomputación en subproblemas solapados.

Los **algoritmos greedy** y las **heurísticas** ofrecen un espectro de eficiencia a costa de garantías formales de optimalidad: el greedy es exacto cuando se cumplen sus condiciones; las heurísticas son aproximaciones prácticas para problemas intratables.

El enfoque de **plantillas de abstracción** proporciona una herramienta cognitiva poderosa: permite identificar la estructura invariante de cada técnica y distinguirla de los elementos específicos del problema, facilitando la transferencia de conocimiento y reduciendo errores de diseño.

Finalmente, la **tabla hash** es una de las estructuras de datos más omnipresentes, fundamentada en la función de hashing que mapea claves a índices de forma eficiente, y empleada internamente por muchos de los algoritmos estudiados como estructura auxiliar de soporte.